CE152: Object-Oriented Programming

Introduction

Dr. Sebastian Halder

CSEE

2026

Introduction

People

Module Supervisor:
Dr Sebastian Halder

Second Module Supervisor:
Dr Hossein Anisi

- GLAs:
  - Saba Zahedieh
  - Bertram Wheen

Course outline

- Solving problems using *object-oriented programming (OOP)*
- Writing programs using Java
- There will be a Lecture each week in which I will demonstrate code and answer questions for each topic
- There are videos for each topic ***which you should watch before the Lecture***
- Check the Moodle which videos to watch when
- Please note that ***not all the content will be repeated*** during the Lecture

Lab sessions

- Labs are Tuesdays (14:00-16:00) and Thursdays (14:00-16:00)
  - Should find the day in your timetable
- ***Please go to the labs and ask for help whenever you need it***

Support classes and other ways to obtain support

- There will be a *voluntary* support classes on Thursdays (11:00-13:00) in *every week* of the term
- In the Moodle under *Module resources* you will find a *Forum for questions*
- You can drop by in my *Support Hour Mondays (14:00-16:00)* or *email me* for another appointment
- Email me if you have a question!

Moodle resources

- On the Moodle you will find:
  - slides for each unit (all available)
  - videos for all the content (all available)
  - lab scripts (release schedule see next slide)
  - assignment task sheet (will be released end of February)
  - source code (this is important because sometimes the slides will only show fragments)
  - a day after the Lecture a link to the video, code and slides of that lecture
- You will find additional links to resources in the Moodle

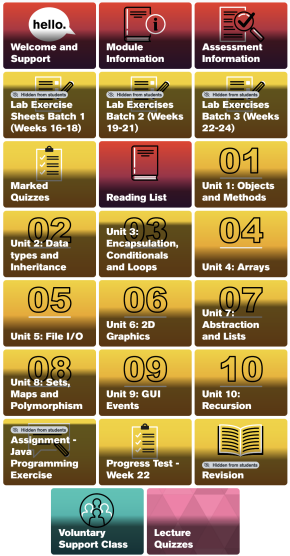
Assessment

- ***This is a coursework only module***
- Three marked lab exercises (8% each)
  - **Handed out** in Week 16, Week 19 and Week 22
  - **Marked in** the labs in Week 17/18, Week 20/21 and Week 23/24
  - ***You need to be in the lab to demonstrate the exercises otherwise you will receive a mark of zero***
  - There is **no Faser submission**
- Progress Test (20%) in Week 22
- Programming Assignment at the end of the term (40% demonstration + 10% code)
  - **Submitted** to Faser and **demonstrated** in Week 31
  - ***You need demonstrate the assignment otherwise you will receive a mark of zero for the demonstration***
  - Marked weekly Moodle quizzes (6% total)

Lab exercise release schedule

- On Moodle you will find a section called **Lab Exercise Sheets**
- In Week 16, Week 19 and Week 22 new lab exercises will be released
- These will be released in batches of three (so in each of the three weeks mentioned above you will receive two briefs with unmarked exercises and one with marked exercises)
- You **can only demonstrate in weeks 17/18, 20/21, 23/24** because the demonstrations take up a lot of resources and there should be time for other questions as well

Location of Lab Exercise Sheets on Moodle



The assignment / “Java Programming Exercise”

- May cover all topics of the module
- Will be released on Moodle at the end of February
- You should start working on it as early as possible and make a concise plan of what to complete when
- Needs to be **demonstrated and submitted to Faser**

Marked quizzes

- Are all in the same section on Moodle called "Marked Quizzes"
- All quizzes must be completed in the week following the corresponding Lecture (e.g. Quiz 1 has to be completed by Sunday 11:59 PM Week 17)
- All answers within one quiz must be correct
- You have unlimited attempts as long as the quiz is open
- The quizzes enable you to get **feedback each week if you are keeping up with the material**
- My recommendation is to attempt the quiz as early as possible and then to revise and attempt again
- Each quiz is worth **0.6% of the module marks**
- Examples: you have all answers in all 10 quizzes correct: 6%; you have all answers in Quiz 1, 3, 9 correct: 1.8%; you have 50% correct in all 10 quizzes: 0%

Demonstrations

- The demonstrations during the term are three check points to assess your learning
- There are two reason for demonstrations (as opposed to marked Faser submissions or tests)
  1. To provide you with **personalised feedback**
  2. To assess you fairly in times of LLMs such as ChatGPT
- During the term you will have to complete the programming task as outlined in the brief (see brief for the first marked lab on Moodle) in front of a supervisor
- Due to the reduced number of GLAs these need to be tasks that can be completed in a few minutes (see brief)
- **Please note:** during demonstration of the final assignment ("Java Programming Exercise") you must be able to explain all of your code instead of writing it!

How much work is needed for all of this?

Amount of work needed (1)

- A 15-credit module such as CE152 will need about 150 h of work
- Considering the time needed for the lectures and labs:

	Weekly (h)	Total (h)
Lecture	2	20
Labs	2	20

- There are 110 h left!

Amount of work needed (2)

- Considering the time for marked exercises:

	Weekly (h)	Total (h)
Exercise 1	8	8
Exercise 2	8	8
Exercise 3	8	8
Assignment	40	40

- There are still 46 h left!

Amount of work needed (3)

- You could use this time for revision and practice

	Weekly (h)	Total (h)
Revision	2.3	23
Coding	2.3	23

Study advice

- Unfortunately these are average values
- This means that **you individually may have to invest more time** (or in some cases also less time) to pass the module
- Learning a new programming language requires **a lot of effort**
- Most importantly it takes **a lot of practice**
- Only **listening to me will not will not give you the ability to program Java**
- You must reserve time **each week to study and practice independently** to keep up with the content
- If you do not start right away it will become harder and ***harder to do so as the term progresses***

The importance of learning Java/OOP in this term

- Even though it is **challenging to learn Java in this term it will only get harder in later terms!**
- E.g. **for three compulsory modules in the next Autumn Term you will need Java** (CE203 Application Programming, CE204 Data Structures and Algorithms) or **will need to understand OOP** (CE202 Software Engineering)
- In these modules **there will be no more time to teach you coding** as the focus is on other content
- Thus I cannot emphasise enough that **if you want to avoid struggling in the second year you need to invest the time now!**

Non-CSEE students

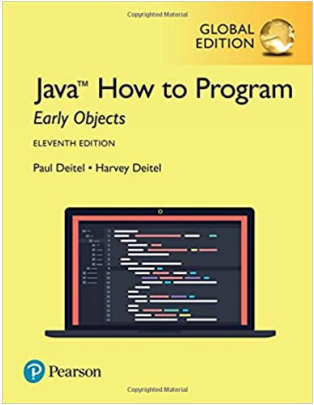
- This is a challenging module
- I expect students to have **prior coding experience (e.g. CE150, CE151)**
- Have a look at https://open.essex.ac.uk/enrol/index.php?id=272 for some of the basics
- Please consider this if you are taking the module voluntarily

What support is there?

Summary of ways to get support

- There are **pre-recorded videos** with slides in PDF format which contain all the theoretical knowledge for the module
- There are the **Lectures** with live-coding as well as organisational announcements, feedback on quizzes etc.
- There are **support classes** with discussion about the content, questions and demonstrations
- There are **labs** with programming exercises and 1-to-1 support by GLAs and module supervisors
- There are **weekly quizzes** for you to check your own progress
- There are two module supervisors who will happily answer any **questions you may have**
- Come to my support hour (Mondays after the lecture 14:00-16:00), email me, arrange separate meetings as often as you want as **I am available for the whole term**
- Your task will be to **take ownership of your own learning** and **practice, practice, practice**

Books



- Have a look at the CE152 reading list if you are interested in a book (there is no required reading)

Tools

- We suggest using an integrated development environment (IDE)
- Most students have preferred IntelliJ
  - The free features are sufficient for this module and are what I use
  - https://www.jetbrains.com/idea/download/other.html
- You can use another environment if you prefer but we may not be able to help you with problems and please make sure to check the requirements on the current marked exercise brief
- You will have to submit the Java Programming Exercise as an IntelliJ project that will run on the lab machines "out-of-the-box"
- See "Getting started instructions" in the first lab for further instructions on how to use IntelliJ

Email

- I will often send important information to the `ce152@essex.ac.uk` mailing list
- If you did not already receive emails via this list please contact the School Office
- For example last week I sent an update regarding Moodle
- Please read those emails to ensure you are aware of deadlines or organisational updates and reminders

Online resources

- The University provides access to O'Reilly online courses:
  - https://learning.oreilly.com/beta-search/?q=java&type=course
  - The website also provides access to many O'Reilly books
- Other options:
  - Free courses: W3Schools, Freecodecamp
  - Commercial sites: Codecademy, Edureka, Hackerrank, Coding Game, Code Wars
  - Programming challenges: Project Euler, Open Kattis
  - Uncountable YouTube channels

Collaboration

- Collaboration is an essential part of academic life
- "Two heads are better than one"
- For example:
  - You can discuss the requirements of assignments with your fellow students
  - You can discuss the content of the slides
  - You can exchange references to books or websites you found helpful

Anything you should NOT do?

Plagiarism (1)

- Please remember cooperation is important and substantially different from plagiarism
  - ***Under no circumstances copy program code from another student or the internet or any other source or let another student copy from you***
  - ***This would be plagiarism and considered an academic offence***
  - For more information on academic offences https://www.essex.ac.uk/student/exams-and-coursework and https://www.essex.ac.uk/student/academic-skills/academic-integrity
- ***All submissions*** are checked for similarities
- Please do not use ChatGPT or similar tools to complete your course work
- *Of course you can use ChatGPT or other models to answer question and to tutor yourself*

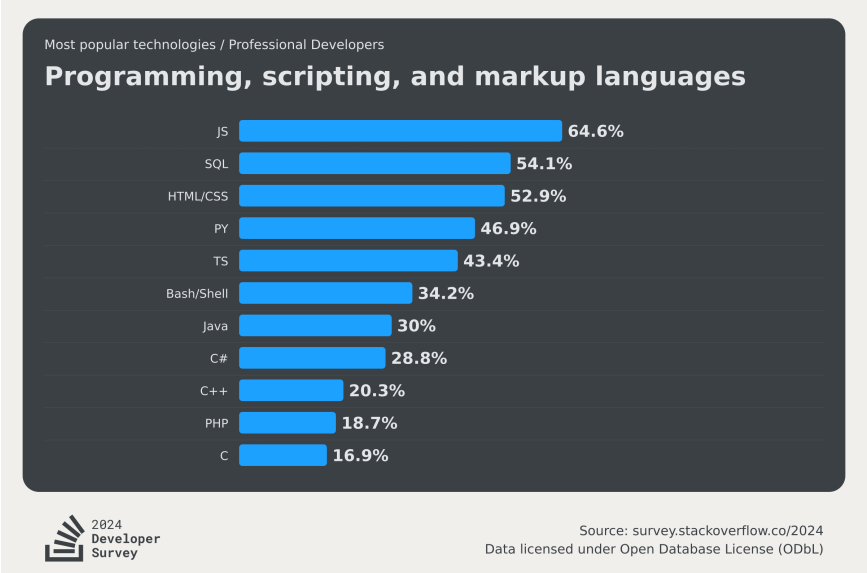
Plagiarism (2)

- It would be easy for you to get the answers for the weekly quiz from a Discord group or another source
- You will get the marks but **you will be cheating yourself** out of the opportunity to check your progress
- Understand that **your goal is not just to pass the module but to learn the material and required skills**
- **Do NOT end up in the second term without the required Java programming skills!**

Doing nothing

- **The most important thing you should NOT do: nothing**
- **Many students fail** the module each year and this does not need to happen
- It frustrates me because I am convinced **anyone can learn Java**
- It requires a lot of effort but **it is easiest if you start now!**

So why Java?


Most popular technologies / Professional Developers
Programming, scripting, and markup languages
JS 64.6%
SQL 54.1%
HTML/CSS 52.9%
PY 46.9%
TS 43.4%
Bash/Shell 34.2%
Java 30%
C# 28.8%
C++ 20.3%
PHP 18.7%
C 16.9%
2024 Developer Survey
Source: survey.stackoverflow.co/2024
Data licensed under Open Database License (ODbL)

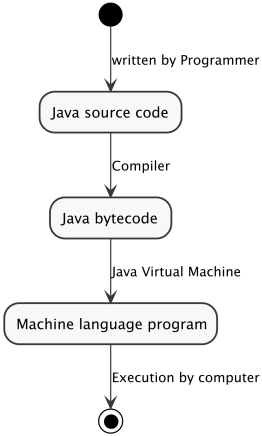
Some history

- The first **Java Development Kit (JDK)** was released on the 23rd of May 1995
- This was seven months after the release of the first Netscape Navigator and 12 years before the first iPhone
- Google did not exist
- Originally developed by Sun Microsystems but since acquired by Oracle
- As seen on the previous slide it is still one of the most used programming languages
- Java is used on servers in the internet, in desktop applications, mobile phones and to control household devices, cars, elevators and even on chip cards
- We will see why on some of the next slides
- The latest version, released September 2025, is Java 25
- Java 8, 11, 17, 21 and 25 are long-term support (LTS) versions (as of January 2026; updates are available longer for these versions)
- Java 8 and above are suitable for the module

Why Java in CE152?

- Java is **class-based** and **object-oriented**
- In **class-based** languages you define the class as a blue print
- Creating something on the basis of the blue print is called **instantiating an object/creating an instance of a class**
- Please find additional information on this principle in the videos, the first lab exercise and the demonstrations following the presentation of these slides

Why Java in general?



- Java is intended to let developers **write once, run anywhere**
  - Java is compiled into so-called **bytecode**
  - This **bytecode** can run on the **Java virtual machine (JVM)**
  - The JVM can run on various architectures (e.g. Android phones)
- The JVM adds **security** as it is essentially a sandbox that prevents the code from doing evil things on the computer it is running on

Any other reasons to use Java?

- Java is a **robust language** in comparison to e.g. C++ as Java does automatic garbage collection which prevents memory leaks
- Java is a (relatively) **easy-to-learn language** (compared to C++ there are no pointers and operators cannot be overloaded; harder compared to Python as there is static typing and a stricter syntax)
- Java supports **encapsulation** which refers to the use of access modifiers (such as **private**) to prevent the unintended access and modification of data (of course other languages also support this)

What is the difference to Python?

Feature	Python	Java
Language	Interpreted	Compiled and Interpreted
Typing	Dynamic	Static
Applications	Scientific, numeric, machine learning, image processing	Web applications, GUIs, enterprise solutions, embedded systems
Syntax	Less code without types and brackets	More code with rigorous syntax
Performance	Slower due to dynamic typing	Faster bytecode
Stability	No syntax test during runtime	High stability
Development	Fast due to simplicity	Slower
Learning	Easy	Harder

- See https://www.imaginarycloud.com/blog/python-vs-java/amp/ for more details

What will we do in CE152?

Currently on Moodle

- **It is very important that you watch the videos**
- All videos are part of the learning goals and may be part of the assessment
- Please ask for help in the lecture, the support class (!), the labs, the forum, via email or in my academic support hour
- **Please invest time to understand the videos each week because the content will become more complex towards the end of the term**

Unit 1 (Week 16)

- Objects and Classes (40 min)
  - Defining **classes** in Java
  - Instantiating **Objects** using constructors
- The main method (10 min)
  - Adding a **main** method and launching a Java program
- Methods (25 min)
  - Defining **methods**
  - Using **return types** and **parameters**

Unit 2 (Week 17)

- Data types (50 min)
  - Differentiating **primitives** and **Objects**
  - Overview of data types in Java
- Inheritance (45 min)
  - Using **Inheritance** to derive one class from another

Unit 3 (Week 18)

- Encapsulation (15 min)
  - Using **Encapsulation** to reduce coupling between classes
  - Choosing access levels with the keywords **public, private** and **protected**
- Conditionals (25 min) and Loops (20 min)
  - Controlling the flow of our code using **conditionals and loops**
- Up to Unit 3 may appear in **Marked Lab Exercise 1 (8% of total mark)**

Unit 4 (Week 19)

- Arrays Part 1 (45 min) and Part 2 (45 min)
  - Storing multiple instances of a class in an **array**
  - Creating **arrays** with **two dimensions** or more

Unit 5 (Week 20)

- Exceptions and File I/O (50 min)
  - Dealing with errors during runtime using **exceptions**
  - Reading data from **files**

Unit 6 (Week 21)

- 2D Graphics Part 1 (50 min) and Part 2 (60 min)
  - Displaying **graphics** using Java
  - **Animation** of two-dimensional shapes
- Up to Unit 6 may appear in **Marked Lab Exercise 2 (8% of total mark)**
- Up to Unit 6 may appear in **Progress test (20% of total mark)**

Unit 7 (Week 22)

- Abstraction (45 min)
  - Using **abstract classes** and **interfaces**
  - Using the interface **Comparable** to make our classes sortable
- Lists (60 min)
  - Introduction of **dynamic data structures** in Java

Unit 8 (Week 23)

- Sets and Maps (25 min)
  - Storing data **without duplicates (Sets)** and using **Objects as keys (Maps)**
- Polymorphism (55 min)
  - Storing variables with **different types in the same data structure**

Unit 9 (Week 24)

- GUI events and design (60 min)
  - Responding to **mouse** and **keyboard** events
  - Using **layouts** to arrange elements of GUIs
- Up to Unit 9 may appear in **Marked Lab Exercise 3 (8% of total mark)**

Unit 10 (Week 25)

- Recursion (40 min)
  - Implementing algorithms and mathematical functions using **recursion**
- All Units may appear in **Java Programming Exercise (10% of total mark for submission and 40% for demonstration)**

Summary

Summary

- Java may seem a bit complicated at first but I hope that at the end of the term you will agree that it is great language to learn Object-Oriented Programming
- In the second half of the term the programs we write in this module ***will quickly increase in complexity***
- This makes ***it very important that you follow the lecture from the start***
- This is why there are ***voluntary support lectures on Thursdays LTB03 11:00***
- You will benefit from knowing Java because it is one of the most used programming languages
- Please install IntelliJ before the labs on Tuesda/Thursday!
- Now we will learn how to create and define Objects!

Thank you and let's write some code!